

Database Basics: ACID, DDL, DML, and SQL Practice

1. What is a Database?

A database stores information in an organized way. In a relational database such as MySQL, PostgreSQL, or SQL Server, data is stored in **tables**.

A table contains:

- **Columns:** Define what type of information is stored.
- **Rows:** Represent individual records.

Example:

id	name	email
1	Alice	alice@example.com
2	Bob	bob@example.com

2. ACID Properties

ACID properties ensure that database transactions are processed safely and reliably.

A **transaction** is a group of database operations treated as one unit.

For example, transferring money between two bank accounts requires:

1. Deducting money from one account.
2. Adding money to another account.

Both operations must succeed together.

Atomicity

Atomicity means **all operations in a transaction succeed, or none of them are applied**.

Suppose \$100 is transferred from Alice to Bob.

```
START TRANSACTION;
```

```
UPDATE accounts
```

```
SET balance = balance - 100  
WHERE id = 1;
```

```
UPDATE accounts  
SET balance = balance + 100  
WHERE id = 2;
```

```
COMMIT;
```

If the second update fails, the database should undo the first update.

```
ROLLBACK;
```

This prevents Alice from losing money without Bob receiving it.

Consistency

Consistency means a transaction must leave the database in a **valid state**.

For example, suppose an account balance is not allowed to become negative.

```
balance DECIMAL(10, 2) CHECK (balance >= 0)
```

A transaction that violates this rule should be rejected.

Consistency is maintained through:

- Primary keys
 - Foreign keys
 - Unique constraints
 - Check constraints
 - Data types
 - Application rules
-

Isolation

Isolation means transactions running at the same time should not interfere with each other incorrectly.

For example, two users may try to purchase the final item in stock at the same time.

Without proper isolation, both transactions might see:

```
stock = 1
```

and both might complete the purchase.

Database isolation helps ensure that concurrent transactions behave correctly.

Durability

Durability means that after a transaction is committed, the result remains saved even if the server crashes.

```
COMMIT;
```

After the commit succeeds, the data should not disappear because of a restart or power failure.

ACID Summary

Property	Meaning
Atomicity	All operations succeed or all are undone
Consistency	Data remains valid
Isolation	Concurrent transactions do not interfere incorrectly
Durability	Committed data remains permanently stored

3. DDL and DML

SQL commands are grouped into different categories.

DDL: Data Definition Language

DDL commands define or change the database structure.

Common DDL commands:

Command	Purpose
CREATE	Creates a database or table
ALTER	Changes an existing table
DROP	Deletes a table or database

Command	Purpose
TRUNCATE	Removes all rows from a table

Examples:

```
CREATE TABLE users (...);
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
DROP TABLE users;
```

DDL mainly works with the **structure** of the database.

DML: Data Manipulation Language

DML commands work with the data stored inside tables.

Common DML commands:

Command	Purpose
INSERT	Adds new rows
UPDATE	Changes existing rows
DELETE	Removes rows

Examples:

```
INSERT INTO users (name, email)
VALUES ('Alice', 'alice@example.com');
UPDATE users
SET name = 'Alice Smith'
WHERE id = 1;
DELETE FROM users
WHERE id = 1;
```

SELECT is sometimes placed under **DQL**, or Data Query Language, because it reads data without changing it.

4. Creating a Database

```
CREATE DATABASE shop_db;
```

Select the database:

```
USE shop_db;
```

5. Creating a Table

Let us create a `products` table.

```
CREATE TABLE products (  
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  category VARCHAR(50) NOT NULL,  
  price DECIMAL(10, 2) NOT NULL,  
  stock INT UNSIGNED NOT NULL DEFAULT 0,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Column Explanation

Column	Meaning
<code>id</code>	Unique identifier for each product
<code>name</code>	Product name
<code>category</code>	Product category
<code>price</code>	Product price
<code>stock</code>	Available quantity
<code>created_at</code>	Time the row was created

Important rules:

- `PRIMARY KEY` uniquely identifies each row.
- `AUTO_INCREMENT` automatically generates IDs.
- `NOT NULL` means a value is required.
- `DEFAULT` provides a value when none is supplied.
- `DECIMAL(10, 2)` supports monetary values such as 1299.99.

6. Inserting Values

Insert one product:

```
INSERT INTO products (  
  name,  
  category,  
  price,
```

```

        stock
    )
VALUES (
    'Wireless Mouse',
    'Electronics',
    25.99,
    20
);

```

Insert multiple products:

```

INSERT INTO products (
    name,
    category,
    price,
    stock
)
VALUES
    ('Mechanical Keyboard', 'Electronics', 79.99, 15),
    ('Office Chair', 'Furniture', 199.99, 8),
    ('Standing Desk', 'Furniture', 349.99, 5),
    ('Notebook', 'Stationery', 4.50, 100),
    ('Pen Set', 'Stationery', 8.99, 50);

```

7. Viewing the Data

Select every column and row:

```

SELECT *
FROM products;

```

Select only specific columns:

```

SELECT name, price
FROM products;

```

Expected result:

name	price
Wireless Mouse	25.99
Mechanical Keyboard	79.99
Office Chair	199.99

8. Altering the Table

Suppose we want to store whether a product is available.

```
ALTER TABLE products
ADD COLUMN available BOOLEAN NOT NULL DEFAULT TRUE;
```

Now the table contains an additional `available` column.

Rename a column:

```
ALTER TABLE products
RENAME COLUMN available TO is_available;
```

Change a column definition:

```
ALTER TABLE products
MODIFY COLUMN name VARCHAR(150) NOT NULL;
```

Remove a column:

```
ALTER TABLE products
DROP COLUMN is_available;
```

Be careful when dropping columns because the data inside them is also removed.

9. Updating Data

Change the price of a product:

```
UPDATE products
SET price = 29.99
WHERE id = 1;
```

Update multiple columns:

```
UPDATE products
SET
    price = 74.99,
    stock = 18
WHERE name = 'Mechanical Keyboard';
```

Increase every product price by 10%:

```
UPDATE products
SET price = price * 1.10;
```

Important Warning

Always check the `WHERE` condition before running an update.

This updates one row:

```
UPDATE products
SET stock = 0
WHERE id = 3;
```

This updates every row:

```
UPDATE products
SET stock = 0;
```

10. Basic Queries

Filter with `WHERE`

Find electronics products:

```
SELECT *
FROM products
WHERE category = 'Electronics';
```

Find products costing more than \$50:

```
SELECT name, price
FROM products
WHERE price > 50;
```

Use Multiple Conditions

Find furniture products with stock greater than five:

```
SELECT *
FROM products
WHERE category = 'Furniture'
    AND stock > 5;
```

Find products belonging to either electronics or stationery:

```
SELECT *
FROM products
WHERE category = 'Electronics'
    OR category = 'Stationery';
```

Sort with `ORDER BY`

Sort from cheapest to most expensive:

```
SELECT name, price
FROM products
ORDER BY price ASC;
```

Sort from most expensive to cheapest:

```
SELECT name, price
FROM products
ORDER BY price DESC;
```

Limit the Result

Return the three most expensive products:

```
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 3;
```

Search with `LIKE`

Find product names beginning with Office:

```
SELECT *
FROM products
WHERE name LIKE 'Office%';
```

Find product names containing Desk:

```
SELECT *
FROM products
WHERE name LIKE '%Desk%';
```

The `%` symbol represents any number of characters.

Use `IN`

Instead of writing several `OR` conditions:

```
SELECT *
FROM products
WHERE category IN ('Electronics', 'Furniture');
```

Use Aliases with `AS`

```
SELECT
    name AS product_name,
    price AS product_price
FROM products;
```

Aliases make output column names easier to understand.

11. Aggregate Queries

Aggregate functions calculate values from multiple rows.

Count Products

```
SELECT COUNT(*) AS total_products
FROM products;
```

Find the Average Price

```
SELECT AVG(price) AS average_price
FROM products;
```

Find the Highest and Lowest Prices

```
SELECT
    MAX(price) AS highest_price,
    MIN(price) AS lowest_price
FROM products;
```

Calculate the Total Stock

```
SELECT SUM(stock) AS total_stock
FROM products;
```

12. Grouping Data

Count the number of products in each category:

```
SELECT
    category,
    COUNT(*) AS product_count
FROM products
GROUP BY category;
```

Calculate the average price of each category:

```
SELECT
    category,
    AVG(price) AS average_price
FROM products
GROUP BY category;
```

Filter grouped results using HAVING:

```
SELECT
    category,
    COUNT(*) AS product_count
FROM products
GROUP BY category
HAVING COUNT(*) >= 2;
```

WHERE filters rows before grouping. HAVING filters groups after grouping.

13. Deleting Data

Delete one product:

```
DELETE FROM products
WHERE id = 5;
```

Delete products with no stock:

```
DELETE FROM products
WHERE stock = 0;
```

Delete every row while keeping the table:

```
DELETE FROM products;
```

A faster way to remove all rows is:

```
TRUNCATE TABLE products;
```

Delete the entire table:

```
DROP TABLE products;
```

The difference is:

- **DELETE** removes selected rows.
 - **TRUNCATE** removes all rows but keeps the table.
 - **DROP** removes both the table and its data.
-

14. Recommended Practice Order

A good practice sequence is:

```
CREATE DATABASE shop_db;

USE shop_db;

CREATE TABLE products (...);

INSERT INTO products (...);

SELECT * FROM products;

ALTER TABLE products ...;

UPDATE products
SET ...
WHERE ...;

SELECT ...
FROM products
WHERE ...
ORDER BY ...;

DELETE FROM products
WHERE ...;
```

Before running an **UPDATE** or **DELETE**, first use the same condition in a **SELECT** query:

```
SELECT *
FROM products
WHERE id = 3;
```

Once you confirm that the correct rows are selected, run the update or delete:

```
UPDATE products
SET stock = 10
WHERE id = 3;
```

This habit helps prevent accidental data loss.

